

Лабораторна робота №4

Семенюка Владислава Григоровича

Група ІПЗ-21

Варіант 7

Код консольної програми

```
using System.Text;

Console.WriteLine("Lab4");
Console.WriteLine("Student: Semeniuk Vlad");
Console.WriteLine("Group: IPZ-21");
Console.WriteLine("Variant: 7");
Console.WriteLine();

string divider = "-----";

Console.Write("Enter your text: ");
string text = Console.ReadLine() ?? "";

if (string.IsNullOrEmpty(text))
{
    Console.WriteLine("Your text is invalid.");
    return;
}

int letterCount = 0;
int digitCount = 0;
int whiteSpaceCount = 0;
int punctuationCount = 0;
```

```
int upperCount = 0;
int lowerCount = 0;

foreach (char symbol in text)
{
    if (char.IsLetter(symbol))
    {
        letterCount++;
    }
    if (char.IsDigit(symbol))
    {
        digitCount++;
    }
    if (char.IsWhiteSpace(symbol))
    {
        whiteSpaceCount++;
    }
    if (char.IsPunctuation(symbol))
    {
        punctuationCount++;
    }
    if (char.IsUpper(symbol))
    {
        upperCount++;
    }
    if (char.IsLower(symbol))
    {
        lowerCount++;
    }
}
```

```
Console.WriteLine(divider);
Console.WriteLine($"User's text: {text}");
Console.WriteLine($"Length: {text.Length}");
Console.WriteLine($"The first symbol: '{text[0]}' , index:
0");
Console.WriteLine(
    $"The last symbol: '{text[^1]}' , " +
    $"index {text.Length - 1}");

Console.WriteLine($"Letter count: {letterCount}");
Console.WriteLine($"Digits count: {digitCount}");
Console.WriteLine($"White space count:
{whiteSpaceCount}");
Console.WriteLine($"Punctuation count:
{punctuationCount}");
Console.WriteLine($"Upper count: {upperCount}");
Console.WriteLine($"Lower count: {lowerCount}");
Console.WriteLine(divider);

// White spaces don't create empty words.
string[] words = text.Split(
    ' ',
    StringSplitOptions.RemoveEmptyEntries);
Console.WriteLine($"Words count: {words.Length}");

for (int i = 0; i < words.Length; i++)
{
```

```

        Console.WriteLine($"{i + 1}) {words[i]} -
{words[i].Length}");
    }

    Console.WriteLine(divider);

    // Example of an individual exercise.
    Console.WriteLine($"Starting text: {text}");

    StringBuilder result = new StringBuilder();

    for (int i = 0; i < text.Length; i++)
    {
        if (i % 2 == 0)
        {
            result.Append(text[i]);
        }
    }

    string changedText = result.ToString();
    Console.WriteLine($"Result: {changedText}");
    Console.WriteLine(divider);

```

Додаткове завдання на фреймворку Avalonia

```

<Application xmlns="https://github.com/avaloniaui"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="AvaloniaApplication1.App"

```

```
        RequestedThemeVariant="Default">
        <!-- "Default" ThemeVariant follows system
theme variant. "Dark" or "Light" are other available
options. -->

    <Application.Styles>
        <FluentTheme />
    </Application.Styles>
</Application>
<?xml version="1.0" encoding="utf-8"?>
<assembly manifestVersion="1.0"
xmlns="urn:schemas-microsoft-com:asm.v1">
    <!-- This manifest is used on Windows only.
        Don't remove it as it might cause problems with
window transparency and embedded controls.
        For more details visit
https://learn.microsoft.com/en-us/windows/win32/sbscs/app
lication-manifests -->
    <assemblyIdentity version="1.0.0.0"
name="AvaloniaApplication1.Desktop"/>

    <compatibility
xmlns="urn:schemas-microsoft-com:compatibility.v1">
        <application>
            <!-- A list of the Windows versions that this
application has been tested on
                and is designed to work with. Uncomment the
appropriate elements
                and Windows will automatically select the most
compatible environment. -->
```

```

        <!-- Windows 10 -->
        <supportedOS
Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}" />
    </application>
</compatibility>
</assembly>
<Window xmlns="https://github.com/avaloniaui"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:d="http://schemas.microsoft.com/expression/blend/20
08"

xmlns:mc="http://schemas.openxmlformats.org/markup-compat
ibility/2006"
        mc:Ignorable="d" d:DesignWidth="800"
d:DesignHeight="450"
        x:Class="AvaloniaApplication1.MainWindow"
        Title="Lab4_Semeniuk_07_Avalonia">

    <StackPanel Margin="20" Spacing="10"
HorizontalAlignment = "Center" VerticalAlignment="Top">

        <TextBlock x:Name="HeaderText" Text="Text
processing" FontSize="24"/>
        <TextBlock x:Name="StartingText" Text="Text
processing" FontSize="16" Margin="0 12 0 0"/>
        <TextBox x:Name="ProcessInput" Width="200"
HorizontalAlignment="Left"/>

        <StackPanel Orientation="Horizontal" Spacing="20">

```

```
        <Button Content="Process"
Click="OnProcessButtonClicked"
HorizontalAlignment="Center"/>

        <Button Content="Clear"
Click="OnClearButtonClicked"
HorizontalAlignment="Center"/>

    </StackPanel>

    <TextBlock x:Name="ResultText" Text="Result"
FontSize="16" Margin="0 12 0 0 "/>

    <Border BorderBrush="Gray" BorderThickness="1"
CornerRadius="4">

        <ScrollViewer x:Name="OutputScrollViewer"
            Height="250"

HorizontalScrollBarVisibility="Auto"

VerticalScrollBarVisibility="Auto">

            <TextBlock x:Name="OutputTextBlock"
                TextWrapping="Wrap"
                Margin="8" />

        </ScrollViewer>
    </Border>

    <TextBlock x:Name="StatusText" Text="Status:
Ready" FontSize="16" Margin="0 12 0 0 "/>

</StackPanel>
</Window>
```

```
using Avalonia.Controls;
using Avalonia.Interactivity;

namespace AvaloniaApplication1;

public partial class MainWindow : Window
{
    private readonly TextAnalyzer _analyzer = new();

    public MainWindow()
    {
        InitializeComponent();

        private void OnProcessButtonClicked(object? sender,
RoutedEventArgs e)
        {
            string inputText = ProcessInput.Text?.Trim() ??
string.Empty;

            if (string.IsNullOrEmpty(inputText))
            {
                StatusText.Text = "Status: Input cannot be
empty.";

                OutputTextBlock.Text = string.Empty;
                return;
            }

            // Run analysis and format result

```



```

        var results = _analyzer.Analyse(inputText);
        OutputTextBlock.Text =
            _analyzer.FormatReport(inputText, results);
        StatusText.Text = "Status: Success.";
    }

    private void OnClearButtonClicked(object? sender,
RoutedEventArgs e)
    {
        ProcessInput.Clear();
        OutputTextBlock.Text = string.Empty;
        StatusText.Text = "Status: Ready.";
    }
}

using System;
using System.Net.Mime;
using System.Text;

namespace AvaloniaApplication1;

public class TextAnalysisResult
{
    public int Length { get; init; }
    public char FirstChar { get; init; }
    public char LastChar { get; init; }
}

```

```

    public int Letters { get; init; }
    public int Digits { get; init; }
    public int WhiteSpaces { get; init; }
    public int PunctuationCount { get; init; }
    public int UpperCount { get; init; }
    public int LowerCount { get; init; }
    public string[] Words { get; init; } =
Array.Empty<string>();

    public string FilteredEvenChars { get; init; } =
string.Empty;

}

public class TextAnalyzer
{
    private const string Divider =
"-----\n";

    public TextAnalysisResult Analyse(string inputText)
    {
        int letters = 0, digits = 0, whiteSpaces = 0,
punctuations = 0, upperCase = 0, lowerCase = 0;

        foreach (char symbol in inputText)
        {
            if (char.IsLetter(symbol)) letters++;
            if (char.IsDigit(symbol)) digits++;
            if (char.IsWhiteSpace(symbol)) whiteSpaces++;
            if (char.IsPunctuation(symbol))
punctuations++;

            if (char.IsUpper(symbol)) upperCase++;

```

```

        if (char.IsLower(symbol)) lowerCase++;
    }

    var evenChars = new StringBuilder(inputText.Length
/ 2 + 1);

    for (var i = 0; i < inputText.Length; i += 2)
evenChars.Append(inputText[i]);

    return new TextAnalysisResult()
    {
        Length = inputText.Length,
        FirstChar = inputText[0],
        LastChar = inputText[^1],
        Letters = letters,
        Digits = digits,
        WhiteSpaces = whiteSpaces,
        PunctuationCount = punctuations,
        UpperCount = upperCase,
        LowerCount = lowerCase,
        Words = inputText.Split(' ',
StringSplitOptions.RemoveEmptyEntries),
        FilteredEvenChars = evenChars.ToString(),
    };
}

public string FormatReport(string rawInput,
TextAnalysisResult stats)
{
    var sb = new StringBuilder();

```

```
sb.AppendLine("Lab4");
sb.AppendLine("Student: Semeniyuk Vlad");
sb.AppendLine("Group: IPZ-21");
sb.AppendLine("Variant: 7");
sb.AppendLine(Divider);
sb.AppendLine($"User's text: {rawInput}");
sb.AppendLine($"Length: {rawInput.Length}");
sb.AppendLine($"The first symbol: '{rawInput[0]}' ,
index: 0");
sb.AppendLine($"The last symbol: '{rawInput[^1]}' ,
" +
                $"index {rawInput.Length - 1}");
sb.AppendLine($"Letter count: {stats.Letters}");
sb.AppendLine($"Digits count: {stats.Digits}");
sb.AppendLine($"White space count:
{stats.WhiteSpaces}");
sb.AppendLine($"Punctuation count:
{stats.PunctuationCount}");
sb.AppendLine($"Upper count: {stats.UpperCount}");
sb.AppendLine($"Lower count:
{stats.LowerCount}\n");

sb.AppendLine(Divider);
for (var i = 0; i < stats.Words.Length; i++)
{
    sb.AppendLine($" {i + 1} {stats.Words[i]} -
{stats.Words[i].Length}");
}
sb.AppendLine(Divider);
```

```

        sb.AppendLine($"Starting text: {rawInput}");
        sb.AppendLine($"Result: (even index)
{stats.FilteredEvenChars}");
        sb.AppendLine(Divider);

        return sb.ToString();
    }
}

using Avalonia;
using System;

namespace AvaloniaApplication1;

class Program
{
    // Initialization code. Don't use any Avalonia,
third-party APIs or any
    // SynchronizationContext-reliant code before AppMain
is called: things aren't initialized
    // yet and stuff might break.
    [STAThread]
    public static void Main(string[] args) =>
BuildAvaloniaApp()
        .StartWithClassicDesktopLifetime(args);

    // Avalonia configuration, don't remove; also used by
visual designer.
    public static AppBuilder BuildAvaloniaApp()
        => AppBuilder.Configure<App>()

```

```
        .UsePlatformDetect()  
#if DEBUG  
        .WithDeveloperTools()  
#endif  
        .WithInterFont()  
        .LogToTrace();  
}
```

ШІ використовувався тільки для ресьорчу.